

Software Design Document

MorningBriefing.AI

“Turn information into clarity, Turn your day into a story”

Team Members

Name	Email	Role
Minsoo Ahn	minsoo.ahn@stonybrook.edu	Project Lead / Full-Stack
Sungmin Choo	sungmin.choo@stonybrook.edu	Backend / Infrastructure
Dahyun Kwon	dahyun.kwon@stonybrook.edu	Frontend / UI/UX

Version: 2.0
Date: April 30, 2026

Table of Contents

Table of Contents.....	2
1. Introduction.....	4
1.1 Product Description.....	4
1.2 Scope.....	4
1.3 Users.....	4
1.4 User Feedback.....	5
1.5 Existing Alternatives.....	5
1.6 Definitions.....	5
1.7 References.....	6
2. Requirements.....	7
2.1 Functional Requirements.....	7
2.1.1 Briefing.....	7
2.1.2 Journaling and Daily Question.....	7
2.1.3 Calendar and Todo Management.....	8
2.1.4 Widget System and Dashboard.....	8
2.1.5 User Settings and Personalization.....	8
2.1.6 Data Persistence and Caching.....	9
2.2 Use Cases.....	9
2.3 User Interfaces.....	11
Layout Overview.....	11
TopNav.....	12
Modals.....	12
2.4 Non-Functional Requirements.....	12
3. System Architecture.....	14
3.1 Overview.....	14
3.1.1 Frontend Stack.....	14
3.1.2 Backend / Infrastructure Stack.....	14
3.1.3 External APIs and Third-Party Services.....	15
3.1.4 Design Pattern.....	15
3.1.5 Technology Stack Rationale.....	15
3.1.6 Current Active Edge Functions.....	15
3.2 Data Design.....	17
3.2.1 UML Class Diagram.....	17
3.2.2 Table Design Notes.....	17
3.3 UML Sequence Diagrams.....	18
3.3.1 Diagram 1.....	18

3.3.2 Diagram 2.....	18
3.4 API Design.....	20
3.4.1 API Route Table.....	20
3.5 Deployment.....	23
3.5.1 Deployment Architecture.....	23
3.5.2 Environment Variables.....	23
3.5.3 Rationale.....	23
3.6 Code Conventions.....	24
3.6.1 Style Guides.....	24
3.6.2 Naming Conventions.....	24
3.6.3 Comment Standards.....	24
3.6.4 Additional Practices.....	24
4. Schedule.....	25
4.1 Gantt Chart.....	25
4.2 Task Dependencies.....	26
5. Contributions.....	27

1. Introduction

1.1 Product Description

Morning Briefing.AI is a desktop web app that helps people get off to a more efficient start to their day. Every morning, it offers customized AI-based briefings that summarize the weather, stock market data, the latest news, and your Google calendar in one simple summary. Instead of checking multiple apps before 9 a.m., you now only need to open one tab.

In addition, the app automatically handles daily diary entries. At midnight, it collects all the activities that the user did that day (such as completed tasks, calendar schedules, and answers to simple daily questions) and creates diary entries without having to write them by the user himself. Key features include:

- Custom AI Morning Briefing: Groq LLM generates a personalized daily summary on dashboard load. Tone (formal/casual) and length (short/medium/long) are user-configurable.
- Diary Cards: Rotating daily reflection prompts with free-form response input. Responses feed both the midnight diary synthesis and the Persona keyword engine.
- Integrated Calendar and To-Do Management: It is synchronized with Google Calendar and comes with a built-in to-do list. Users check what they actually complete, and only those items are recorded in their diary.
- Automated Midnight Journal Generation: At midnight, the system automatically gathers data from the day to create a structured diary entry consisting of the [Macro Events] and [Personal Records] sections.
- 30-Day Keyword Decay: Adaptive Persona engine that scores user interests from Diary Card responses and decays them over 30 days, ensuring briefings always reflect current relevance.

1.2 Scope

MorningBriefing.AI targets desktop web browsers (Chrome 85+, Edge 85+) at viewport widths of 1200px and above. The application is designed for individual use; team sharing and public collaboration are out of scope.

External integrations: Google Calendar, Google Tasks, Google Fitness, OpenWeatherMap, Twelve Data (stocks), Tavily (news/trends), Groq LLM.

Out of scope for current version: mobile apps, social sharing, diary emotion analysis, fitness tracker beyond step/sleep/heart-rate read, email/push notification summaries.

1.3 Users

Primary: Office workers and students who use desktop computers for daily work, already use Google Calendar, and want to minimize information-gathering time in the morning.

Secondary: People interested in passive daily journaling who want an automated record of their day without manual writing effort.

1.4 User Feedback

From early user interviews, AI-generated content was perceived as too generic. This directly motivated the Persona system: customizable tone, length, and keyword-driven interest profiles. The most critical feedback window is after one week of automated diary usage, when users can judge the quality and relevance of the generated entries.

We plan to gather feedback through informal interviews with a few early users. While we haven't done any official user testing yet, based on the initial conversations, the most common concern is that AI diaries can feel too general or inhumane.

1.5 Existing Alternatives

The closest competitor is Rosebud (rosebud.ai), an AI-powered journaling app with daily reflection prompts. Rosebud is mobile-only and requires manual input per session. MorningBriefing.AI differentiates on two axes: (1) desktop-first workflow integration, and (2) fully automated diary generation from real usage data. That is, no manual writing is required.

1.6 Definitions

Custom Briefing	Personalized AI summary generated by Groq LLM based on the user's Persona settings and current macro data.
Macro Events	Real-time external data: weather, financial indices (KOSPI, Nasdaq, USD/KRW), trending news.
Diary Card	Left-panel widget presenting a daily rotating reflection question and free-form text input.
Midnight Synthesis	Automated process at 00:00 that aggregates day data and calls Groq LLM to generate the daily journal entry.
Persona	User-defined AI voice profile: tone, length, interests. Drives briefing style and smart widget personalization.
SmartKeyword	User-defined keyword (up to 5) that generates a personalized AI content card in the SmartWidgetContent component.
ScoreLog	Database table tracking quantitative engagement scores per keyword per day, used by the 30-day Decay Formula.
Widgets	Modular UI components on the dashboard: WeatherWidget, StocksWidget, TrendsWidget, HealthWidget, NewsWidget, SmartWidgetContent.

Zero-Entry Journaling	The automated diary generation model — no manual writing required from the user.
[MB_META]	App-specific metadata stored in Google Tasks notes field for extended task properties (repeat, dueDate, completedDates).

1.7 References

- CSE 416: Functional Requirements Document, March 23, 2026
- Supabase Documentation — <https://supabase.com/docs>
- Groq API Documentation — <https://console.groq.com/docs>
- Twelve Data API Documentation — <https://twelvedata.com/docs>
- Tavily Search API Documentation — <https://docs.tavily.com>
- Google Calendar API — <https://developers.google.com/calendar/api>
- Google Tasks API — <https://developers.google.com/tasks>
- Google Fitness REST API — <https://developers.google.com/fit/rest>

2. Requirements

2.1 Functional Requirements

Format: 'The User can [action]' or 'The System shall [action].' Updates since the SRS: (1) dynamic AI synthesis pipeline with Persona-based keyword decay; (2) Google Tasks integration added as full CRUD source; (3) UI redesigned to 2-column layout.

2.1.1 Briefing

- REQ-BR-001: The System shall generate a personalized AI morning briefing on each dashboard load using current weather, financial indices, trending news, and the user's Google Calendar events.
- REQ-BR-002: The User can set the briefing tone (formal or casual) and length (short / medium / long) in Settings.
- REQ-BR-003: The System shall automatically show a full expanded briefing modal on the user's first dashboard open each calendar day after 00:00. Triggers once per day.
- REQ-BR-004: The User can disable the automatic daily briefing modal in Settings; the BriefingWidget remains accessible at all times.
- REQ-BR-005: The System shall generate Smart Widget content using Groq LLM based on up to 5 user-defined keywords, with a 20-minute minimum cache TTL.

2.1.2 Journaling and Daily Question

- REQ-JR-001: The System shall display a new rotating reflection question in the DiaryCard each day at 00:00, with no repeated question within a 30-day window.
- REQ-JR-002: The User can type a free-form response and optional memo note in the DiaryCard; the response auto-saves on blur.
- REQ-JR-003: The User can view and edit the current day's diary entry; past entries are read-only after the calendar day ends.
- REQ-JR-004: The System shall trigger Midnight Synthesis at 00:00, collecting checked todos, completed calendar events, daily question response, memo, and a macro data snapshot.
- REQ-JR-005: The System shall call Groq LLM to generate a structured, factual journal entry with [Macro Events] and [Personal Records] sections. Emotional language is excluded.
- REQ-JR-006: The System shall show a journal preview modal after generation. If dismissed without action, the entry auto-saves after 30 seconds.
- REQ-JR-007: The User can manually trigger diary generation (Generate / Regenerate Diary) at any time, independent of the midnight automated process.

- REQ-JR-008: The System shall support optional PIN protection for diary access. When PIN is enabled, a PINModal authentication step is required before DiaryPanel or DiaryListModal is shown.

2.1.3 Calendar and Task Management

- REQ-CA-001: The System shall sync Google Calendar events into the CalendarWidget via Google OAuth, displayed in chronological order with timezone support.
- REQ-CA-002: The User can create, edit, and delete Google Calendar events including recurring events (RRULE series), guests, Google Meet link, rich-text description, location (via Google Places), visibility, busy/free status, and reminders.
- REQ-CA-003: The System shall sync Google Tasks via Google OAuth. The User can create, edit, complete, and delete tasks across multiple Google Task Lists.
- REQ-CA-004: App-specific task metadata (repeat pattern, Deadline/dueDate, completedDates) is stored in the Google Tasks notes field as [MB_META] JSON. Recurring task completion is tracked per occurrence; edits/deletes apply to the full series.
- REQ-CA-005: The User can check off calendar events and todos; only checked items are included in the nightly journal synthesis.

2.1.4 Widget System and Dashboard

- REQ-WS-001: The System shall render the dashboard in a 2-column layout: Left panel (Calendar + Diary); Right panel (AI Briefing + Widgets carousel). Minimum viewport: 1200px.
- REQ-WS-002: The User can reorder Standard Widgets via drag-and-drop; the order is persisted to Supabase and influences AI briefing content priority.
- REQ-WS-003: The User can toggle visibility of any Standard Widget in Settings. Fixed components (BriefingWidget, DiaryCard, CalendarWidget) cannot be hidden.
- REQ-WS-004: The System shall display real-time data: WeatherWidget (10-min TTL), StocksWidget (5-min TTL, KOSPI/NASDAQ/S&P500/USD-KRW), TrendsWidget/NewsWidget (20-min TTL via Tavily).

2.1.5 User Settings and Personalization

- REQ-US-001: The User can toggle dark/light mode from TopNav or Settings; selection persists across sessions.
- REQ-US-002: The User can choose clock style: digital (HH:MM:SS), analog, or full date-info format.
- REQ-US-003: The User can set a custom data priority ranking for Standard Widgets via drag-and-drop in Settings.
- REQ-US-004: The System shall maintain a 30-day Keyword Decay Formula for Persona. Each keyword score is weighted by $(30 - \text{elapsed_days}) / 30$. Scores at zero are purged. Keyword sources: Diary Card responses and Personal Q&A.

- REQ-US-005: The User can configure diary language (en/ko) and PIN settings in the Settings > Diary tab.
- REQ-US-006: The System shall support full UI localization for en and ko, including all panel copy, date formats, repeat/reminder labels, and error states.

2.1.6 Data Persistence and Caching

- REQ-DS-001: The System shall persist all user data to Supabase with Row-Level Security (RLS) enforcing per-user data isolation.
- REQ-DS-002: The System shall use a multi-layer cache: Supabase api_cache table (1-hour TTL default), localStorage fallback, and module-scope memory cache for short-TTL data (geolocation ~5 min).
- REQ-DS-003: All third-party API keys shall be stored exclusively in Supabase Secrets Manager. No keys shall be exposed in the frontend bundle or browser DevTools.

2.2 Use Cases

The five use cases below cover the most important user interactions.

UC-01: Read the Morning Briefing	
Actor	Registered User
Priority	Essential
Scenario	1. The user opens the Morning Briefing.AI dashboard in a browser. 2. If it is the first opening time since midnight and the modal is active, the system displays the full AI briefing modal (10 to 15 lines). 3. Users read the briefing, including weather, market data, major news, and today's schedule. 4. The user closes the modal and checks the 3-line summary in BriefingWidget. 5. The user scrolls through the middle column for more information from WeatherWidget, StocksWidget, and TrendsWidget.
Extensions	• 2a. If API calls fail: cached data or mock data is displayed. Briefing continues to be generated using available data. • 2b. If the user disables automatic modal in Settings: Modal does not appear. The user reads manually by clicking on the BriefingWidget detail icon.

UC-02: Respond to the Daily Journaling Question	
Actor	Registered User
Priority	Essential

Scenario	1. The user sees the DiaryCard with today's reflection questions in the left column. 2. Users click on the text field and type their answers. 3. The system automatically saves the answer to the Supabase on blur 4. Users can optionally add notes to the Memo field. 5. At midnight (00:00 a.m.), the system includes both the response and memo in the [Personal Records] section of the journal.
Extensions	• 3a. If the Supabase write fails: the system retries up to 3 times, then saves to localStorage with a non-blocking error notice. • If the user has not answered before 00:00, the journal entry notes 'No daily reflection recorded.'

UC-03: Manage Tasks and Calendar Events

Actor	Registered User
Priority	Essential
Scenario	1. The user opens the CalendarWidget and reviews calendar events for today or a selected date. 2. The user selects a date to open the EventPanel and TaskPanel for that date. 3. In the EventPanel, the user can create, edit, or delete Google Calendar events. 4. In the TaskPanel, the user can create a task, edit task details, and mark a task as completed. 5. The system syncs event changes to Google Calendar and task changes to Google Tasks in real time. 6. At diary generation time, completed tasks for that date are included automatically, and calendar events for that date are collected as part of the daily context. Users check today's Google Calendar events in the CalendarWidget.
Extensions	• 1a. If Google Calendar or Google Tasks is temporarily unavailable, the system may show cached local data and display an error or reconnect flow. The displayed data may not be fully up to date. • 4a. If the user marks a task as incomplete again before diary generation, it is excluded from that day's synthesis.

UC-04: Automatic Midnight Journal Generation (System)

Actor	System (Automated Process)
Priority	Essential
Scenario	1. At 00:00 the system starts the Midnight Synthesis process.

	- The system collects all the checked tasks and completed calendar events in the last 24 hours. - The system searches for responses and notes to today's diary questions. - The system takes a snapshot of macro data (weather, stocks, and major news). - The system sends structured context payloads to Groq LLM and requests fact-based journal entries. - Groq returns a journal (approximately 300 to 500 words) that contains the [Macro Events] and [Personal Records] sections. - The system displays preview modals. Users can review, edit, or confirm saves. - Items are saved in the Supabase as 'completed' or 'user_edited'.
Extensions	1a. If a user's browser closes at 00:00: Synthesis runs the next time a session opens. Modals appear at dashboard loading. 5a. If Groq times out (>30s): the system will try again. If it still fails, the raw data will be saved as a draft and the user will be notified. 7a. When the user closes the modal without confirmation: the input will be saved automatically after 30 seconds.

UC-05: Review Past Journal Entries	
Actor	Registered User
Priority	Expected
Scenario	- The user clicks the History icon on the DiaryCard. - The Diary Modal opens and displays the entries for the current date by default. - The user uses the date selector to move to the previous date. - System retrieves and displays the selected date's question, response, memo, and generated journal. - Past entries are read-only. Only the current day's entry is editable.
Extensions	3a. If no entry exists for the selected date: modal shows 'No record found for this date.' - If the user tries to edit a past entry: edit controls are disabled with a tooltip explaining that past entries are locked.

2.3 User Interface

Update since SRS: The layout has been redesigned from a 1:3:3 three-column structure to a cleaner 2-column layout. This change was driven by user feedback that dense multi-column dashboards (similar to Naver or Microsoft Edge) create visual noise and reduce focus.

Layout Overview

- **Left Panel:** CalendarWidget (month view, event/task panels, DatePanelContainer)
- **Right Panel:** BriefingWidget (AI summary, detail modal trigger) + DiaryCard (today's reflection prompt, memo field) + Standard Widgets carousel (WeatherWidget, StocksWidget, TrendsWidget, NewsWidget, HealthWidget, SmartWidgetContent) — swipeable with trackpad gesture support.

TopNav

- Left: User profile icon with profile/settings access.
- Center: Clock (digital HH:MM:SS / analog / full date — user selection) and search placeholder.
- Right: QuickLinks (user-defined bookmarks) and dark/light theme toggle.

Modals

- Daily Briefing Modal: Auto-shown on first open each day. Full 10–15 line AI analysis. Dismissible via close button or background click.
- DiaryPanel / DiaryListModal: Date-navigable diary view. Current day is editable; past days are read-only. Optional PIN authentication gate.
- PINModal: Setup stage (first time) or Verify stage (returning). Locks diary content behind 4-digit PIN stored in localStorage.
- EventPanel / TaskPanel: Google Calendar event and Google Tasks CRUD panels rendered as date-anchored portals.
- Settings Modal: Tabs for Theme, Clock, AI Briefing, Diary (PIN + language), Smart Widget Keywords, Widget Visibility, Data Priority.

2.4 Non-Functional Requirements

Performance

- REQ-NFR-P01: Dashboard full load under 3 seconds on a 4G network (ApiCache layer as primary optimization).
- REQ-NFR-P02: Each widget renders within 500ms of data arrival.
- REQ-NFR-P03: AI features (briefing generation, diary synthesis) complete within 10 seconds. Loading indicator required during wait.
- REQ-NFR-P04: Drag-and-drop widget reordering maintains 60 FPS on standard laptops.

Scalability

- REQ-NFR-SC01: System handles at least 10,000 concurrent users without degradation.
- REQ-NFR-SC02: Backend is stateless. Supabase Edge Functions scale horizontally without configuration.

Availability and Reliability

- REQ-NFR-AV01: 99.5% uptime target.
- REQ-NFR-AV02: External API failures (AI, stocks, weather) must not crash the app. Fallback to cached data or mock values.
- REQ-NFR-AV03: Failed requests are retried up to 3 times before surfacing an error state.

Security

- REQ-NFR-SE01: All client-server communication encrypted via HTTPS.
- REQ-NFR-SE02: Supabase RLS enforces per-user data isolation at the database level.
- REQ-NFR-SE03: All third-party API secrets stored exclusively in Supabase Secrets Manager / Vercel environment variables. Never exposed in the browser.

Accessibility

- REQ-NFR-AC01: WCAG 2.1 Level AA compliance.
- REQ-NFR-AC02: Minimum 4.5:1 text contrast ratio against background.
- REQ-NFR-AC03: Full keyboard navigation (Tab, Enter, Space) without mouse dependency.

Platform & Compatibility

- REQ-NFR-BR01: Fully functional on latest Chrome and Edge.
- REQ-NFR-BR02: Desktop-only. Minimum supported viewport: 1200px width.

3. System Architecture

3.1 Overview

MorningBriefing.AI follows a Component-Based Architecture on the frontend and a Serverless Infrastructure on the backend. The system is composed of three major layers:

- Frontend (React SPA): Handles all UI rendering, local state management, and user interactions. Deployed to Vercel.
- Backend (Supabase Edge Functions): TypeScript-based serverless functions acting as a secure API gateway between the client and all external data sources. No secrets are exposed to the browser.
- Database (Supabase PostgreSQL): Managed relational database with Row-Level Security. Stores user settings, diary entries, caching layer, and keyword scores.

3.1.1 Frontend Stack

React	18 — Component-based UI library. Functional components + Hooks throughout.
Vite	6 — Build tooling and dev server. Fast HMR, optimized production bundles.
Tailwind CSS	3 — Utility-first CSS. Consistent design tokens, dark/light mode.
Zustand	Latest — Centralized client state management. Stores: useAuthStore, useDataStore, useDiaryStore, useGoogleCalendarStore, useTodoStore, useWidgetStore, useSettingsStore, useQuickLinksStore.
Framer Motion	Latest — Declarative animation library for widget transitions and modal entry/exit.
@hello-pangea/dnd	Latest — Drag-and-drop widget reordering (fork of react-beautiful-dnd).
i18next / react-i18next	Latest — Full UI localization for en and ko.
lucide-react	Latest — Icon library.

3.1.2 Backend / Infrastructure Stack

Supabase	Cloud — Managed PostgreSQL, Auth (Google OAuth), and Edge Functions runtime.
Edge Functions	TypeScript — 9 serverless functions acting as secure API gateway. Deno runtime.
PostgreSQL	Supabase-managed — Primary data store with RLS policies.

Vercel	Latest — Frontend deployment with CI/CD from GitHub. Environment variable management.
---------------	---

3.1.3 External APIs and Third-Party Services

Groq LLM	AI briefing and diary generation. Chosen over OpenAI for inference speed — latency-critical for briefing generation.
Tavily	AI-optimized news and trends search. Returns LLM-ready structured output; no additional parsing overhead.
Google Calendar API	Two-way event sync with RRULE recurring event support.
Google Tasks API	Two-way task CRUD. App-specific metadata stored in [MB_META] notes field.
Google Fitness REST API	Step count, sleep, heart rate read (scopes: activity, sleep, heart_rate).
OpenWeatherMap	Local weather data for WeatherWidget.
Twelve Data	Financial market indices (KOSPI, Nasdaq, S&P500, USD/KRW).

3.1.4 Design Pattern

Component-Based Architecture (frontend) + **Serverless / BFF** (Backend-for-Frontend) pattern. Edge Functions act as an aggregation and security layer — fetching from multiple external sources and returning a unified, sanitized JSON response to the client. The client never directly contacts external APIs.

3.1.5 Technology Stack Rationale

- React 18 + Vite 6: Maximum UI modularity for independent widget components. Vite's fast HMR accelerates development iteration on a complex multi-widget dashboard.
- Supabase: Eliminates backend server management overhead. Auth, database, and Edge Functions in one platform with first-class RLS support — essential for per-user data isolation.
- Zustand over Redux: Lower boilerplate for per-domain stores. Each feature (diary, calendar, settings) maintains its own store slice while sharing a common Supabase client.
- Groq over OpenAI: Groq's throughput directly translates to a snappier briefing experience. Latency is the primary constraint for morning-use AI features.
- Tavily over generic search: Returns pre-structured, LLM-optimized content — feeds directly into the Groq prompt pipeline without additional parsing.

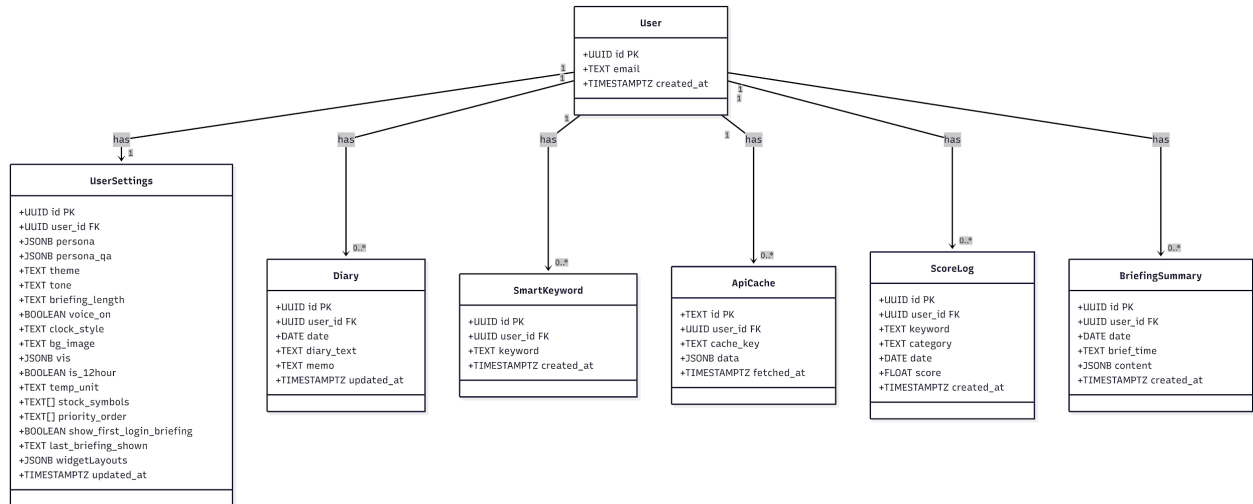
3.1.6 Current Active Edge Functions

events	Google Calendar read/write via server-side OAuth token exchange.
tasks	Google Tasks read/write. Handles [MB_META] metadata parsing.
weather	OpenWeatherMap fetch with location parameter.
stocks	Twelve Data fetch for configured symbol array.
tavily	AI-optimized news/trends search.
fitness	Google Fitness REST read for activity, sleep, heart_rate.
groq	Groq LLM call with persona + context payload.
smart-widget	Keyword-based personalized AI content generation.

3.2 Data Design

3.2.1 UML Class Diagram

All tables are protected by Supabase Row-Level Security (RLS). Users can only access their own rows at the database level. [link](#)



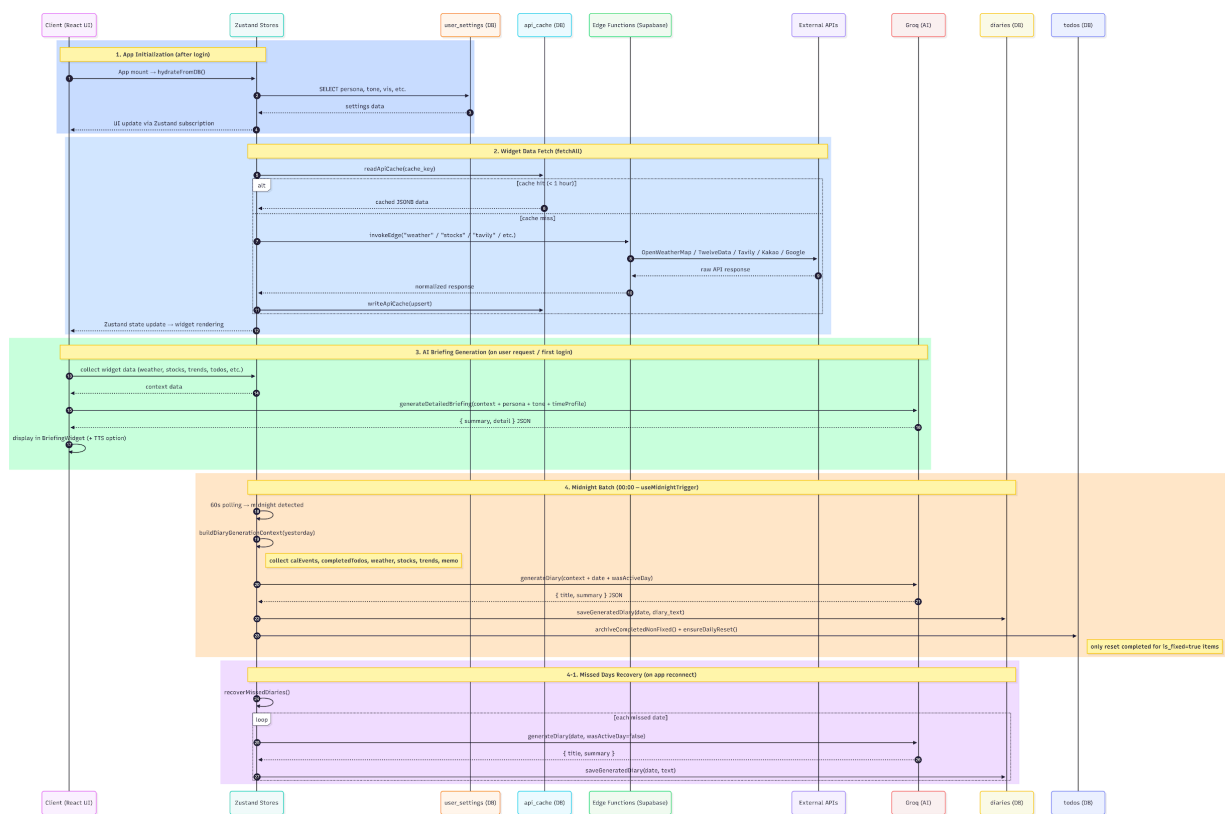
3.2.2 Table Design Notes

- **UserSettings:** The 'brain' of the UI. Stores persona and persona_qa as JSONB objects enabling highly customized AI voice and dashboard state per user in a single 1:1 row.
- **Diary:** Separates ai_generated_diary (raw Groq output) from edited_diary (user-modified version) and memo (free-form daily note). Diary synthesis uses the memo for the same-date entry; briefing generation excludes memo.
- **SmartKeyword + ScoreLog:** Work together for the 30-day Decay Formula. SmartKeyword stores explicit interests; ScoreLog tracks quantitative engagement as FLOAT scores per day. Score weight = $\text{score} \times (30 - \text{elapsed_days}) / 30$. At 0, the keyword is purged.
- **ApiCache:** Middle-tier buffer for external API responses. Stores JSONB payload with fetched_at timestamp. Default TTL: 1 hour. Eliminates redundant calls for weather, stocks, and news within the cache window.
- **BriefingSummary:** Persists generated briefing content by date and brief_time (morning/evening). Allows replay of past briefings.

3.3. UML Sequence Diagrams

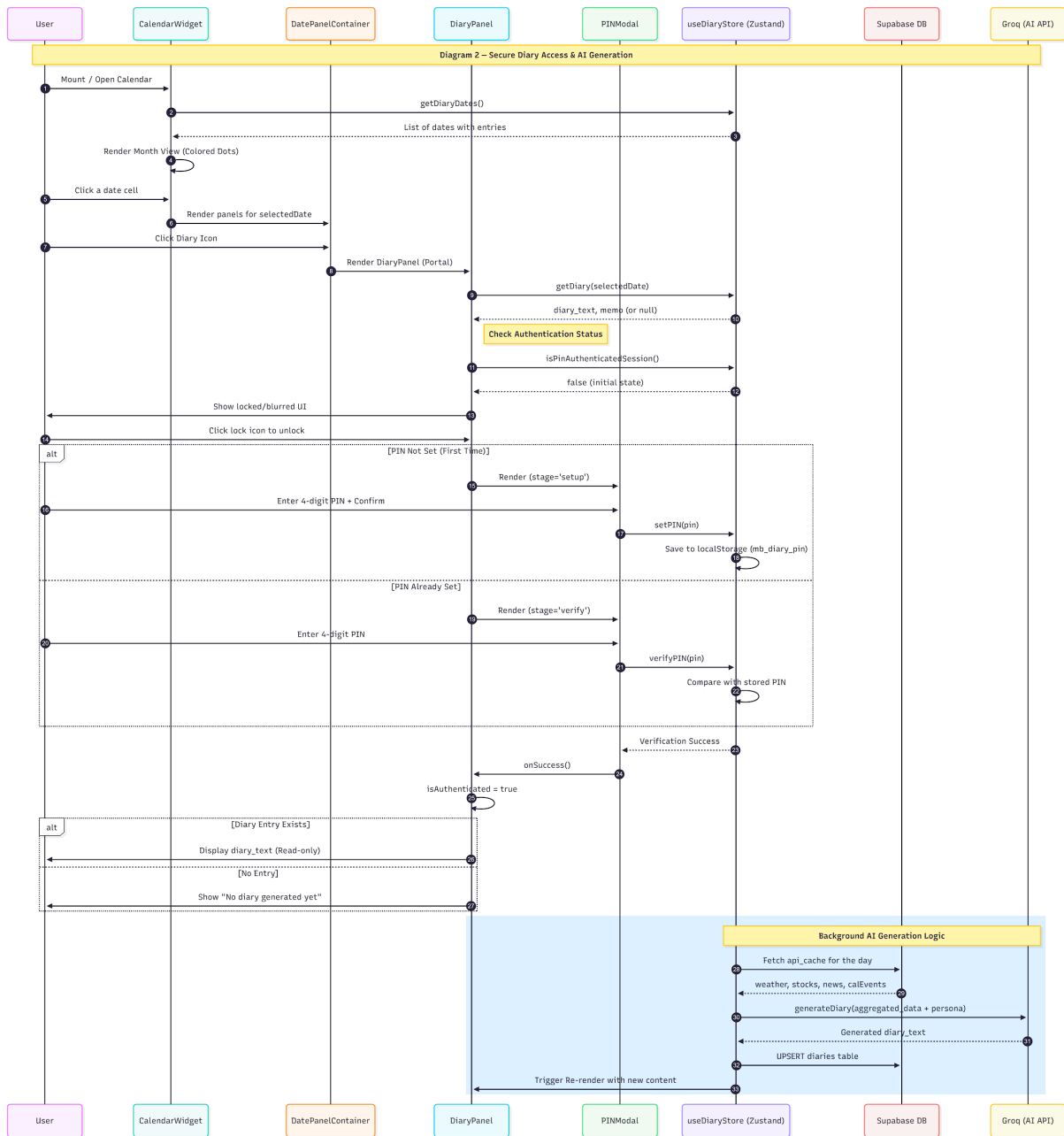
3.3.1 Diagram 1 — App Initialization, Widget Data Fetch, AI Briefing, Midnight Synthesis

This diagram illustrates the full data orchestration of MorningBriefing.AI from app mount through the midnight diary generation cycle. [link](#)



3.3.2 Diagram 2 — Secure Diary Access & AI Generation

This diagram covers the PIN-gated diary access flow and the background AI diary generation process triggered when a user views a date without an existing entry. [link](#)



3.4 API Design

All routes follow a consistent pattern. Every request is authenticated via the user's Supabase JWT. All communication is HTTPS. Edge Functions act as a BFF aggregation layer — no external API secrets ever reach the client.

Base URL pattern: `https://<project>.supabase.co/functions/v1/<name>` | Supabase REST: `https://<project>.supabase.co/rest/v1/<table>`

3.4.1 API Route Table

External Data Endpoints (Edge Functions)

Status	Method	Name	URL (Edge Function)	Input	Output	Flow / Reason	Notes
ACTIVE	POST	Fetch Weather	supabase.co/functions/v1/weather	lat: FLOAT lon: FLOAT	{ temp, feels_like, humidity, condition, icon, city, precipitation, airQuality }	useDataStore.fetchWeather() → weather/index.ts → OpenWeatherMap (current) → OpenWeatherMap Air Pollution API (parallel)	Coords rounded to 0.1 precision for cache reuse. AQI: 1=Good, 2-3=Moderate, 4=Bad, 5=Very Bad. Timeout: 25s.
ACTIVE	POST	Fetch Stocks	supabase.co/functions/v1/stocks	symbols: STR[] (KOSPI, NASDAQ, SP500, USDKRW)	[{ symbol, price, change, changePercent }]	useDataStore.fetchStocks() → stocks/index.ts → Twelve Data API → Fallback: Stooq CSV → Fallback: ER-API (USDKRW)	Symbols normalized uppercase. Stooq fallback: EWY, QQQ, SPY. ER-API fallback timeout: 7s.
ACTIVE	POST	Fetch Trends	supabase.co/functions/v1/tavily	mode: "trends" query: STR	{ results: [{ title, url, content, published_date, image }], answer: STR }	useDataStore.fetchTrends() → tavily/index.ts (mode=trends) → Tavily Search API (advanced, 7 results)	Korean trending keywords. Advanced search depth. Timeout: 25s.
ACTIVE	POST	Fetch News	supabase.co/functions/v1/tavily	mode: "news" lat: FLOAT lon: FLOAT	{ results: [{ title, url, content, published_date, image }], answer: STR }	useDataStore.fetchNews() → tavily/index.ts (mode=news) → Tavily API x2 parallel (local + global news)	10 results total. Location-aware query. Timeout: 25s.

ACTIVE	POST	Fetch / Manage Events	supabase.co/functions/v1/events	token: STR (Google OAuth) action: STR eventId: STR startTime: STR endTime: STR location: STR description: STR	<code>{ id, summary, start, end, location, description, attendees }</code>	useGoogleCalendarStore → token from supabase.auth.getSession() → events/index.ts → Google Calendar API v3	Scope: calendar.events Single endpoint multiplexed by action (list/read/create/update/delete). Timeout: 25s.
ACTIVE	POST	Fetch / Manage Tasks	supabase.co/functions/v1/tasks	token: STR (Google OAuth) action: STR taskId: STR taskListId: STR title: STR notes: STR due: STR	<code>{ id, title, notes, due, status, completed }</code>	useGoogleCalendarStore.fetchTasks() → token from supabase.auth.getSession() → tasks/index.ts → Google Tasks API	Scope: tasks Single endpoint multiplexed by action (list/create/update/delete/move/clearCompleted). App metadata in notes as [MB_META]. Timeout: 25s.
ACTIVE	POST	Fetch Health	supabase.co/functions/v1/fitness	token: STR (Google OAuth)	<code>{ steps: INT, calories: FLOAT, heartRate: FLOAT, sleep: FLOAT }</code>	useDataStore.fetchHealth() → token from supabase.auth.getSession() → fitness/index.ts → Google Fitness API	Scope: fitness.activity.read Returns zeros on 403 (graceful). Aggregated in 86400s bucket. Timeout: 25s.
ACTIVE	POST	Generate AI Brief	supabase.co/functions/v1/groq	prompt: STR system: STR model: STR	<code>{ content: STR }</code>	useDataStore.generateAiBrief() → scoreSignals(weather, stocks, trends, calEvents, todos, persona, timeProfile) → aiService.invokeFunction() → groq/index.ts → Groq API (Llama-3.3-70b-versatile)	temperature=0.4 (briefing). Tone by time of day: morning/lunch/evening. Timeout: 15s. Cached locally in widget.
ACTIVE	POST	Smart Widget Query	supabase.co/functions/v1/smart-widget	lat: FLOAT lon: FLOAT userInput: STR	<code>{ keywords: STR[], suggestions: [...], weather: {...} }</code>	After Q&A submission: → smart-widget/index.ts → Tavily (trending context) → OpenWeatherMap (indoor/outdoor routing)	Groq response_format: {type: "json_object"}. temperature=0.3. Weather context for indoor/outdoor suggestions.

						→ Groq JSON mode (keyword extraction)	
--	--	--	--	--	--	---------------------------------------	--

Supabase REST Endpoints (Database Layer)

Status	Method	Name	URL (Edge Function)	Input	Output	Flow / Reason	Notes
ACTIVE	GET	Read API Cache	supabase.co/rest/v1/api_cache?id=eq.u:{userId}:{cacheKey}	userId: STR (from auth) cacheKey: STR (e.g. "weather_37.5_126.9")	{ data: JSON, fetched_at: DATETIME }	useDataStore.read ApiCache() → Supabase REST (SELECT) → Check 1h TTL vs fetched_at → CACHE HIT → set state → CACHE MISS → real API call	Key: u:{userId}:{cacheKey} TTL: 60 minutes. RLS enforced per user.
ACTIVE	POST	Write API Cache	supabase.co/rest/v1/api_cache	cacheKey: STR data: JSON userId: STR (from auth)	{ error: null }	useDataStore.write ApiCache() → Supabase REST (UPSERT) → Persists after successful API response	UPSERT for concurrent safety. userId from auth context.
ACTIVE	GET	Get User Settings	supabase.co/rest/v1/user_settings?id=eq.{userId}	userId: STR (from auth session)	{ persona: STR, vis: {...}, ...preferences }	useDataStore.fetch All() → Supabase REST (SELECT) → Determines visible widgets and user preferences	RLS enforced. Includes widget visibility and persona settings.
ACTIVE	GET	Get Widget Layouts	supabase.co/rest/v1/widget_layouts?id=eq.{userId}	userId: STR (from auth session)	{ layouts: { bp1: [...], bp2: [...] } }	useDataStore.fetch All() → Supabase REST (SELECT) → Restores grid positions → Extracts smart widget IDs	Stores multi-breakpoint grid positions. Smart widget IDs parsed from layouts.

To view the full API table, follow the [link](#).

3.5 Deployment

3.5.1 Deployment Architecture

Frontend	Vercel — React/Vite SPA. CI/CD integrated with GitHub repository. Automatic preview deployments per pull request.
Backend	Supabase Cloud — Managed PostgreSQL, Auth, and Edge Functions (Deno runtime) in a production-ready environment.
Auth	Supabase Auth with Google OAuth provider. Scopes: calendar, tasks, fitness.activity.read, fitness.sleep.read, fitness.heart_rate.read.
Secrets	All third-party API keys (Groq, Twelve Data, Tavily, OpenWeatherMap) stored in Supabase Secrets Manager. Client-only key: VITE_GOOGLE_MAPS_API_KEY restricted by HTTP referrer in Google Cloud Console.

3.5.2 Environment Variables

Client-side (.env / Vercel environment):

```
VITE_SUPABASE_URL=https://xxxxxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
VITE_GOOGLE_MAPS_API_KEY=AIza...
```

Server-side (Supabase Secrets — never exposed to browser):

```
GROQ_API_KEY
TWELVE_DATA_API_KEY
TAVILY_API_KEY
OPENWEATHER_API_KEY
```

3.5.3 Rationale

Vercel was chosen over GitHub Pages for its seamless CI/CD integration, edge network, and first-class environment variable management — essential for keeping API keys out of the repository. Supabase Cloud handles the entire backend stack (Auth, DB, Functions) without requiring server infrastructure management, directly supporting the Beta release deployment requirement.

3.6. Code Conventions

3.6.1 Style Guides

React / JS	Airbnb JavaScript Style Guide — https://airbnb.io/javascript/react/
TypeScript (Edge Functions)	Google TypeScript Style Guide — https://google.github.io/styleguide/tsguide.html
CSS	Tailwind CSS utility-first conventions. No custom CSS unless Tailwind cannot express the requirement.

3.6.2 Naming Conventions

React Components	PascalCase — BriefingWidget, DiaryPanel, PINModal
Functions / Variables	camelCase — getMidnightSynthesis, fetchAll, buildPersonaContext
Database Columns	snake_case — user_id, diary_text, fetched_at
File Names	camelCase for JS/JSEX — useDataStore.js, aiService.js
Edge Function Dirs	kebab-case — smart-widget/
Constants	SCREAMING_SNAKE_CASE — MAX_RETRY_COUNT, CACHE_TTL_MS

3.6.3 Comment Standards

- Single-line / inline: double forward slash (//) for logic rationale.
- Complex logic (AI prompt construction, 30-day decay formula): JSDoc-style block comments. Provides IDE-level documentation and type hints for teammates.

3.6.4 Additional Practices

- Git: Feature branches off main. PR required for all merges. Branch naming: feature/<name>, fix/<name>.
- No hardcoded secrets in any client file. .env.example committed to repo; actual .env excluded via .gitignore.

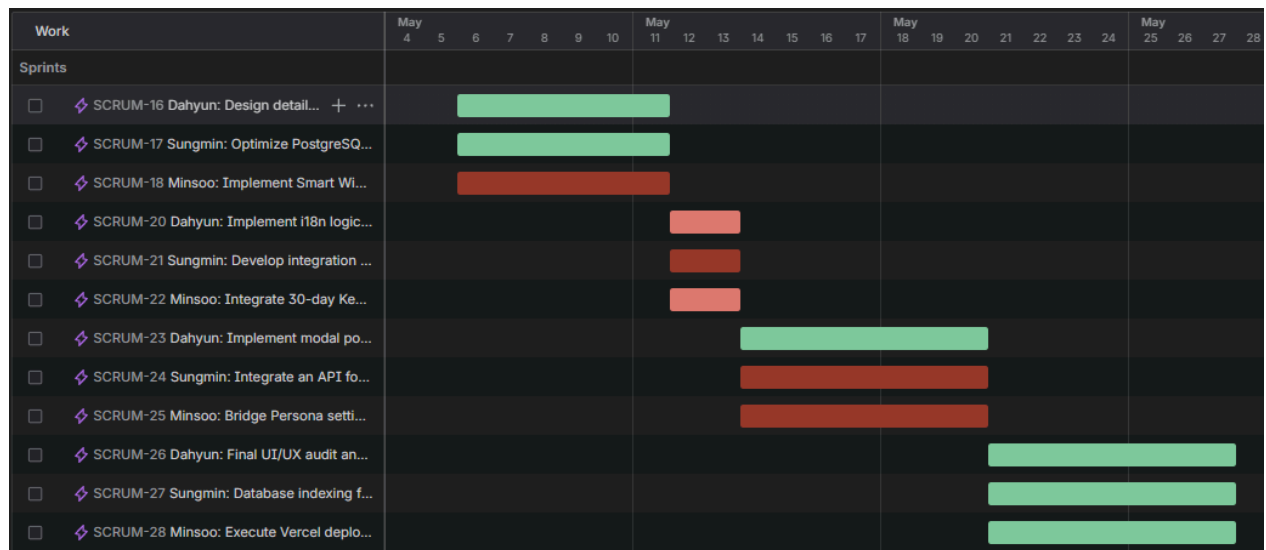
4. Schedule

Four milestones leading to the Beta release on May 27, 2026. Priority levels: High (critical path), Medium (core UX), Low (refinement).

Milestone	Date	Task	Assignee	Priority
M1: May 11	May 11	Finalize Persona Settings & DiaryCard UI wireframes	Dahyun	Low
M1: May 11	May 11	Optimize PostgreSQL schema for activity logging + enforce RLS	Sungmin	Low
M1: May 11	May 11	Smart Widget + Briefing integration	Minsoo	High
M2: May 13	May 13	Multi-language (en/ko) full UI localization	Dahyun	Medium
M2: May 13	May 13	Diary Card response recording & persistence logic	Sungmin	High
M2: May 13	May 13	30-day Keyword Decay formula integration	Minsoo	Medium
M3: May 20	May 20	Modal portals for widget detail views	Dahyun	Low
M3: May 20	May 20	Desktop history API fetch & parse for AI context	Sungmin	High
M3: May 20	May 20	Persona–Groq API bridge + error handling	Minsoo	High
M4: May 27	May 27	Comprehensive UI/UX audit	Dahyun	Low
M4: May 27	May 27	DB indexing + final RLS security audit	Sungmin	Low
M4: May 27	May 27	Vercel production deployment + API monitoring setup	Minsoo	Low

4.1 Gantt Chart

Using a Gantt chart, the project milestones (1-4) can be managed smoothly and tracked visually. Teammates utilized the timeline feature in Jira to organize and monitor the project progress.



4.2 Task Dependencies

- PostgreSQL schema (M1, Sungmin) must complete before Keyword Decay integration (M2, Minsoo).
- Persona–Groq bridge (M3, Minsoo) depends on Persona Settings UI (M1, Dahyun) and Keyword Decay (M2, Minsoo).
- UI/UX audit (M4, Dahyun) depends on Modal portals (M3, Dahyun) being complete.
- Production deployment (M4, Minsoo) depends on security audits (M4, Sungmin).

5. Contributions

Contributions to the Software Design Document:

Name	Email	Main Activities
Minsoo Ahn	minsoo.ahn@stonybrook.edu	Architecture design, API design, deployment strategy, tech stack rationale, security and caching design.
Sungmin Choo	sungmin.choo@stonybrook.edu	Data design (UML class diagram, Supabase schema), Edge Function API spec, schedule planning, sequence diagrams.
Dahyun Kwon	dahyun.kwon@stonybrook.edu	Requirements section, UI/UX description (2-column redesign), code conventions.